

Chord DHT 虚拟节点扩展设计文档

Virtual Node (VNode) Extension Design Specification

版本 v4.0 | 2026年5月30日

字段	内容
文档版本	v4.0
创建日期	2026年5月30日
基于版本	v3.0 (Chord DHT 网络优化扩展设计文档)
文档状态	草案
适用对象	在 v3.0 基础上添加虚拟节点 (VNode) 支持的工程师
适用范围	学习用途, 不适合直接用于生产环境
新增机制	虚拟节点 (VNode)、VNodeProof 签名凭证、锚节点 (Anchor) 概念、共享证书体系、vnode_id 确定性派生

目录

- 第一章：设计背景与目标
- 第二章：核心概念与术语
- 第三章：vnode_id 派生机制
- 第四章：VNodeProof 凭证设计
- 第五章：身份验证协议变更
- 第六章：节点信息 Schema 变更 (NodeInfo v4)
- 第七章：HTTP 路由与共享服务器架构
- 第八章：资源分级共用与维护调度
- 第九章：Tracker API 变更规范
- 第十章：Node API 变更规范
- 第十一章：边缘情况与处理策略 (36 个场景)
- 第十二章：安全攻击面分析
- 第十三章：新增配置参数
- 第十四章：消息时序变更
- 第十五章：实现建议 (Go 语言)
- 第十六章：数据迁移时序

第一章：设计背景与目标

1.1 背景

v1.0 建立了 Chord DHT 的核心骨架：SHA-1 ($m=160$) 环形 ID 空间、手指表、后继列表 ($r=3$)、RESTful HTTP+JSON 协议。v2.0 引入 Ed25519 证书体系，实现了节点身份绑定与请求签名。v3.0 在正确性基础上系统优化了性能，引入自适应维护调度、延迟感知路由、LRU 缓存、动态后继列表与 Region 标签。

然而，v1-v3 均假设一个物理节点只对应一个 Chord ID，这带来以下局限：

- **负载不均衡**：当节点数量较少时，ID 空间分配高度偏斜，部分节点承载数倍于平均水平的键范围。
- **环拓扑脆弱**：少量物理节点构成的小环在节点离开时拓扑剧烈波动。
- **算力浪费**：高配置物理机与低配置物理机在环中地位相同，无法按能力贡献更多 ID 点。
- **一致性哈希均衡性差**：纯 SHA1(uri) 的 ID 分布理论上均匀，但实际节点数量有限时方差极大。

v4.0 引入**虚拟节点 (Virtual Node, 简称 vnode)** 机制，允许每个物理节点在 Chord 环上可选地占据多个位置，同时通过 **VNodeProof 凭证**复用锚节点证书，无需为每个 vnode 单独申请 CA 签发的证书。

1.2 设计目标

1. **可选性**：每个物理节点可配置 vnode 数量 N ($N \geq 0$)， $N=0$ 表示纯锚节点模式，与 v1-v3 完全兼容。
2. **证书复用**：所有 vnode 共享锚节点 (Anchor) 的 Ed25519 证书，通过 VNodeProof 凭证证明归属关系，CA 无需介入 vnode 的生命周期。
3. **确定性 ID**：vnode_id 由 anchor_id 与索引 index 确定性派生，不依赖随机数，可重启后恢复。
4. **协议兼容**：vnode 对外表现为完整的 Chord 节点，其他节点无需感知对方是否为 vnode。
5. **独立状态机**：每个 vnode 维护独立的手指表、后继列表与节点状态机，保证协议正确性。
6. **安全性**：防止 Sybil 攻击 (过度生成 vnode)、ID 位置操纵、VNodeProof 伪造与重放。

1.3 不在设计范围内

- vnode 之间的键迁移优化 (数据感知路由)
- 跨物理机 vnode 迁移 (vnode 始终归属固定 Anchor)
- TLS 传输层加密 (继承 v2.0 限制)
- vnode 的独立 CA 证书申请流程
- 生产级别的 vnode 自动扩缩容

第二章：核心概念与术语

2.1 锚节点（Anchor Node）

物理节点在 Chord 环中的主身份节点，具备：

- 唯一的 `anchor_id = SHA1(host:port)`（继承 v1.0 定义）
- CA 签发的 Ed25519 证书（继承 v2.0 定义）
- 独立的私钥用于签名所有 `VNodeProof`

锚节点本身也是一个正常的 Chord 节点，与 `vnode` 并列参与环路由。

2.2 虚拟节点（Virtual Node / VNode）

由锚节点在本地生成的额外 Chord 节点，具备：

- 由 `anchor_id` 和索引 `index` 确定性派生的 `vnode_id`
- 独立的手指表、后继列表、前驱节点
- 独立的 Chord 状态机（INIT / JOINING / ACTIVE / ISOLATED / LEAVING）
- **无独立证书**，通过 `VNodeProof` 凭证证明与 Anchor 的绑定关系

2.3 VNodeProof 凭证

由 Anchor 私钥签名的结构化凭证，证明：

- 该 `vnode_id` 由该 Anchor 合法派生
- 凭证在有效期内
- 凭证绑定于特定 `anchor_id` 与 `index`，不可挪用

2.4 索引（Index）

`vnode` 在同一 Anchor 下的序号，从 1 开始（0 保留给 Anchor 本身的内部逻辑使用，Anchor 的 Chord ID 直接使用 `anchor_id`，不走 `VNodeProof` 流程）。有效范围：`[1, max_vnodes]`。

第三章：vnode_id 派生机制

3.1 派生公式

```
vnode_id = SHA1( "chord-vnode-v4\n" + anchor_id + "\n" + strconv.Itoa(index) )
```

其中：

- `"chord-vnode-v4\n"` 为固定域分隔前缀（Domain Separation Prefix），防止与其他 SHA1 用途产生碰撞
- `anchor_id` 为 40 位十六进制字符串（不含 `0x` 前缀）
- `index` 为十进制字符串，范围 `[1, max_vnodes]`

- 所有字段以 \n 分隔，拼接为 UTF-8 字节串后输入 SHA1

3.2 设计理由

属性	说明
确定性	相同 anchor_id + index 始终产生相同 vnode_id，节点重启后可恢复身份
不可预选目标	攻击者无法自由选择 vnode_id 落点，index 由 CA 授权范围约束
域分隔	前缀防止与 SHA1(host:port) 产生格式冲突
可验证性	任意节点均可本地重算 vnode_id 以验证 VNodeProof 中声明的一致性

3.3 ID 分布特性

使用上述公式，N 个 vnode 的 ID 分布与直接 SHA1 随机输入等效（SHA1 输出近似均匀），在环上提供良好的负载均衡效果。

第四章：VNodeProof 凭证设计

4.1 数据结构

```
{
  "vnode_id":    "b3f2... (40位十六进制) ",
  "anchor_id":   "a1c9... (40位十六进制) ",
  "index":       2,
  "issued_at":   1748563200,
  "expires_at":  1748649600,
  "anchor_pub":  "Base64(Ed25519公钥, 32字节)",
  "signature":   "Base64(Ed25519签名, 64字节)"
}
```

4.2 签名内容（Canonical Form）

签名覆盖以下字节串（UTF-8，字段间以 \n 分隔）：

```
"chord-vnode-proof-v4\n"
+ vnode_id + "\n"
+ anchor_id + "\n"
+ strconv.Itoa(index) + "\n"
+ strconv.FormatInt(issued_at, 10) + "\n"
+ strconv.FormatInt(expires_at, 10)
```

设计说明

不签名 anchor_pub: anchor_pub 本身通过 CA 证书验证, 签名内容中包含 anchor_id 即可间接绑定。

4.3 颁发流程

1. Anchor 在启动时, 依据配置的 vnode_count 批量生成所有 VNodeProof
2. VNodeProof 由 Anchor 私钥签名, 有效期等于 vnodeproof_ttl (默认 86400 秒)
3. VNodeProof 随 NodeInfo 一并携带, 在 Chord 协议交互中传递给对端验证
4. 到期前 vnodeproof_renew_before (默认 3600 秒) 自动重签, 重签不改变 vnode_id

4.4 验证流程

接收方在处理来自 vnode 的任何请求时:

1. 从请求 Header 中提取 X-Chord-VNode-Proof (Base64 编码 JSON)
2. 解析并检查 expires_at > now
3. 重算 expected_id = SHA1("chord-vnode-v4\n" + anchor_id + "\n" + index) 并与 vnode_id 对比
4. 从本地缓存或随 NodeInfo 携带的 anchor_cert 中提取 anchor 公钥
5. 验证 anchor_cert 对该公钥的 CA 签名 (继承 v2.0 流程)
6. 用 anchor 公钥验证 VNodeProof 签名

注意

任意一步失败, 返回 403 Forbidden , 错误码 INVALID_VNODE_PROOF 。

第五章：身份验证协议变更

5.1 请求签名（继承 v2.0）

vnode 发出的每个 HTTP 请求, 使用**Anchor 的私钥**签名 (与 v2.0 相同的 X-Chord-Signature 机制), 同时在 Header 中附加:

```
X-Chord-Node-ID:      <vnode_id>
X-Chord-Anchor-ID:    <anchor_id>
X-Chord-VNode-Proof: <Base64(VNodeProof JSON)>
```

接收方通过 X-Chord-Anchor-ID 定位 Anchor 证书, 完成签名验证, 再通过 X-Chord-VNode-Proof 验证 vnode 身份。

5.2 纯锚节点请求 (N=0, 向后兼容)

当 X-Chord-VNode-Proof Header 不存在, 且 X-Chord-Anchor-ID 与 X-Chord-Node-ID 相同时, 接收方退回 v2.0 验证路径, 不要求 VNodeProof。

5.3 CA 职责不变

CA 仍仅签发 Anchor 证书, 不感知 vnode 的存在。CA 证书中不包含 max_vnodes 等 vnode 配置, vnode 数量限制通过 Tracker 侧策略与节点自身配置共同约束。

第六章：节点信息 Schema 变更（NodeInfo v4）

6.1 Anchor NodeInfo

```
{
  "node_id":      "a1c9...",
  "uri":          "http://host:port",
  "public_key":   "Base64(Ed25519公钥)",
  "certificate":  { ...v2.0 CA 证书... },
  "region":       "us-east-1",
  "vnodes": [
    {
      "vnode_id": "b3f2...",
      "index":    1,
      "proof":    { ...VNodeProof... }
    },
    {
      "vnode_id": "d7a1...",
      "index":    2,
      "proof":    { ...VNodeProof... }
    }
  ]
}
```

6.2 VNode NodeInfo（对外呈现）

vnode 在 Chord 协议中以普通节点身份与其他节点交互, 其 NodeInfo 仅包含：

```
{
  "node_id":      "b3f2...",
  "uri":          "http://host:port",
  "public_key":   "Base64(Anchor Ed25519公钥)",
  "certificate":  { ...Anchor CA 证书... },
  "region":       "us-east-1",
  "anchor_id":    "a1c9...",
  "vnode_proof":  { ...VNodeProof... }
}
```

说明

当 `anchor_id` 字段存在且非空时，接收方知晓这是一个 `vnode`，触发 `VNodeProof` 验证路径。

第七章：HTTP 路由与共享服务器架构

7.1 单端口多 `vnode` 架构

每个物理节点只监听一个 **TCP 端口**，所有 `vnode` 和 `Anchor` 共享同一个 HTTP Server。通过 URL 路径前缀区分目标节点身份：

```
/chord/node/{node_id}/find_successor
/chord/node/{node_id}/notify
/chord/node/{node_id}/get_predecessor
/chord/node/{node_id}/ping
...
```

其中 `{node_id}` 可以是 `anchor_id` 或任意 `vnode_id`。HTTP 路由层根据 `node_id` 将请求分派到对应的 Chord 状态机实例。

7.2 路由分派器 (Router/Dispatcher)

内部维护一个映射表：

```
type NodeDispatcher struct {
    anchor  *ChordNode
    vnodes  map[string]*ChordNode // vnode_id -> ChordNode
}
```

收到请求时：

1. 从 URL 提取 `node_id`
2. 查找 `vnodes[node_id]`，若未找到则检查是否为 `anchor.id`
3. 若均未找到，返回 `404 Not Found`，错误码 `NODE_NOT_FOUND`
4. 验证 `VNodeProof`（若请求来自 `vnode`）
5. 将请求转发至对应 `ChordNode` 实例处理

7.3 向后兼容路径

保留不含 `node_id` 前缀的旧路径（如 `/find_successor`），默认路由至 `Anchor` 节点实例，供 `v1-v3` 节点互操作。

第八章：资源分级共用与维护调度

8.1 资源分级共用模型

在 v4.0 的多 vnode 架构中，若每个 vnode 完全独立维护所有资源，单台物理机的内存与 CPU 消耗将随 vnode 数量线性增长。以默认 8 个 vnode 为例，手指表、NodeInfo 缓存、RTT 测量、路由缓存等资源若全量复制，将产生 8 倍的冗余开销。为此，v4.0 引入**两层资源分级模型**，将资源按语义分为两类：

层级	名称	共享范围	设计依据
L0	物理机共享层	同一物理机上的所有 vnode 与 Anchor	与具体 Chord ID 无关，语义全局正确
L1	vnode 独立层	单个 vnode 私有	与该 vnode 在环上的位置强绑定，不可共用

8.2 L0 层：物理机共享资源（六类）

8.2.1 HTTP 连接池（Transport Pool）

所有 vnode 与 Anchor 共享同一个 http.Transport 实例，复用 TCP 连接。vnode 数量从 1 增至 8 时，出站连接数量不增加，只是并发请求量增加。

```
sharedTransport := &http.Transport{
    MaxIdleConns:      200,
    MaxIdleConnsPerHost: 20,
    IdleConnTimeout:    90 * time.Second,
}
sharedClient := &http.Client{Transport: sharedTransport, Timeout: rpcTimeout}
```

8.2.2 NodeInfo TTL 缓存

存储已知节点的完整 NodeInfo（含公钥、证书、URI、region、vnode_proof）。手指表（L1）只存轻量的 node_id 字符串引用，需要完整信息时查 L0 缓存。

- 缓存结构：map[node_id] → (NodeInfo, expire_at)，使用读写锁保护
- TTL：shared_nodeinfo_cache_ttl（默认 30 秒），到期后懒惰刷新
- 容量上限：shared_nodeinfo_cache_size（默认 2048 条），超出时按 LRU 淘汰
- 节省：每个 NodeInfo 约 1-2 KB，8 vnode 时节省 7/8 重复存储

8.2.3 RTT 测量缓存与 singleflight 去重

多个 vnode 可能同时对同一目标节点发起 RTT 探测，导致短时间内收到 N 个并发 ping（N = vnode 数量）。L0 层使用 **singleflight** 合并重复测量：同一 targetID 同一时刻只发起一次实际 ping，其余协程等待并复用结果。RTT 缓存 TTL 默认 60 秒。

```
func (r *RTTCache) MeasureRTT(targetID, uri string) (time.Duration, error) {
    r.mu.RLock()
    if e, ok := r.cache[targetID]; ok && time.Since(e.measuredAt) < r.ttl {
        r.mu.RUnlock()
        return e.rtt, nil // 命中缓存，直接返回
    }
```



```

    }
    r.mu.RUnlock()
    // singleflight: 同一 targetID 同一时刻只发起一次实际 ping
    val, err, _ := r.sfGroup.Do(targetID, func() (interface{}, error) {
        return ping(uri)
    })
    if err != nil { return 0, err }
    rtt := val.(time.Duration)
    r.mu.Lock()
    r.cache[targetID] = rttEntry{rtt: rtt, measuredAt: time.Now()}
    r.mu.Unlock()
    return rtt, nil
}

```

8.2.4 路由 LRU 缓存（全局共享）

v3.0 的路由 LRU 缓存在 v4.0 中**提升为物理机级共享**。

正确性依据：Chord 路由具有确定性——拓扑稳定时，对同一 key 的 find_successor 查询无论由哪个 vnode 发起，结果（successor node_id）相同。因此 vnode-1 缓存的路由结果可安全被 vnode-2 复用，8 vnode 时缓存命中率提升约 8 倍。拓扑变化时，缓存 TTL（默认 10 秒）保证最终一致性。

注意

路由缓存命中后，仍需对返回的 successor 执行 VNodeProof 验证（若 successor 是 vnode）。缓存只跳过路由查找网络往返，**不跳过身份验证**。

8.2.5 CRL 缓存（证书吊销列表）

CRL 从 Tracker 周期性拉取（间隔 `crl_refresh_interval`，默认 300 秒），物理机上所有 vnode 共享同一份 CRL 内存副本，无需每个 vnode 独立拉取。CRL 刷新由 Anchor 协程统一负责，刷新后通知所有 vnode 的验证缓存失效。

8.2.6 VNodeProof 验证结果缓存

对来自同一远端 vnode 的重复请求，Ed25519 签名验证（约 50μs/次）可通过结果缓存避免重复计算。

- 缓存键：SHA1(vnode_id + anchor_id + expires_at)，绑定到 proof 的特定版本
- 缓存值：验证结果（valid / invalid），容量 256 条
- Invalidation：Anchor 公钥更新或 CRL 刷新时，清空整个验证缓存

8.3 L1 层：per-vnode 独立资源（轻量化设计）

资源	设计	不可共享的原因
手指表（Finger Table）	160 条 node_id 字符串（轻量化，见 8.4）	每个 vnode 在环上位置不同，finger[i] 指向不同节点
后继列表（Successor List）	动态长度列表，含 anchor_id 标记（多样性约束用）	后继依赖该 vnode 的具体环位置
前驱节点指针（Predecessor）	单个 node_id 引用	前驱依赖该 vnode 的具体环位置

资源	设计	不可共享的原因
Chord 状态机	INIT / JOINING / ACTIVE / ISOLATED / LEAVING	各 vnode 的 Join/Leave 生命周期独立
维护定时器 (stabilize / fix_fingers / check_predecessor)	错峰启动, 独立 ticker	各 vnode 需要独立的维护窗口偏移
ISOLATED 重试状态	退避计数器与重试队列	各 vnode 的网络孤立状态独立

8.4 手指表轻量化详解

原始设计中, 手指表每条存储完整的 NodeInfo (含 URI、公钥、证书、VNodeProof 等), 约 1-2 KB/条, 160 条共约 160-320 KB/vnode, 8 个 vnode 总计约 1.3-2.6 MB 仅用于手指表。v4.0 轻量化后, 每条仅存 node_id (40 字节十六进制字符串), 160 条共 **6.4 KB/vnode**。路由时先查手指表得到 node_id, 再查 L0 NodeInfo 缓存获取完整信息:

```
type FingerTable struct {
    entries [160]string // 每条只存 node_id (40字节), 160条共 6.4KB
    mu      sync.RWMutex
}

func (f *FingerTable) GetNodeInfo(i int, cache *NodeInfoCache) (*NodeInfo, error) {
    f.mu.RLock()
    nodeID := f.entries[i]
    f.mu.RUnlock()
    if nodeID == "" {
        return nil, ErrFingerNotInitialized
    }
    return cache.Get(nodeID) // L0 缓存查找, 命中率通常 >95%
}
```

内存占用对比 (8 vnode 场景):

指标	原始设计 (全量 NodeInfo 存入手指表)	v4.0 轻量化
单 vnode 手指表内存	~160 KB	6.4 KB (仅 node_id)
8 vnode 手指表总内存	~1.28 MB	51.2 KB + L0 缓存 (约 2-4 MB 上限, 全局共享)
8 vnode 总体节省	—	节省约 7× 的 NodeInfo 重复存储

设计权衡

手指表轻量化引入额外的 L0 缓存查找一跳。NodeInfo 缓存命中率通常 >95% (30 秒 TTL 内), 缓存 miss 时触发一次网络请求获取最新 NodeInfo, 代价可接受。

8.5 共享资源层数据结构 (Go 实现)

```

// SharedResources 物理机级共享资源, 由 AnchorNode 持有并传递给所有 VNode
type SharedResources struct {
    HTTPClient    *http.Client    // 共享 HTTP 连接池
    NodeInfoCache *NodeInfoCache // NodeInfo TTL 缓存
    RTTCache      *RTTCache       // RTT 测量缓存 + singleflight
    RouteCache    *RouteLRUCache  // 路由结果 LRU 缓存
    CRLCache      *CRLCache       // 证书吊销列表缓存
    ProofVerCache *ProofVerifyCache // VNodeProof 验证结果缓存
    RateLimiter   *rate.Limiter   // 物理机级入站速率限制
}

// AnchorNode 初始化时创建共享资源, 传递给每个 VNode
func NewAnchorNode(cfg *Config) *AnchorNode {
    shared := &SharedResources{
        HTTPClient:    buildHTTPClient(cfg),
        NodeInfoCache: NewNodeInfoCache(cfg.SharedNodeInfoCacheSize, cfg.SharedNodeInfoCacheTTL),
        RTTCache:      NewRTTCache(cfg.SharedRTTCacheTTL),
        RouteCache:    NewRouteLRUCache(cfg.SharedRouteCacheSize, cfg.SharedRouteCacheTTL),
        CRLCache:      NewCRLCache(cfg.CRLRefreshInterval),
        ProofVerCache: NewProofVerifyCache(cfg.SharedProofVerifyCacheSize),
        RateLimiter:   rate.NewLimiter(rate.Limit(cfg.InboundRateLimit), cfg.InboundRateBurst),
    }
    return &AnchorNode{shared: shared}
}

// VNode 初始化时引用 Anchor 的共享资源 (引用, 不复制)
func NewVNode(anchor *AnchorNode, index int) *VNode {
    return &VNode{
        shared:      anchor.shared, // L0: 引用, 零拷贝
        fingerTable: &FingerTable{}, // L1: 独立手指表 (仅 node_id)
        successors:  []string{},      // L1: 独立后继列表
        predecessor: "",              // L1: 独立前驱指针
        state:       StateINIT,       // L1: 独立状态机
        index:       index,
        anchor:      anchor,
    }
}

```

8.6 维护任务错峰调度

同一物理机上的多个 vnode 若同时触发 stabilize/fix_fingers, 将产生网络突发流量 (Thundering Herd)。采用**错峰启动偏移**, 确保各 vnode 的维护窗口均匀错开:

```

func (v *VNode) startMaintenance(interval time.Duration, jitter int) {
    // vnode_i 的起始偏移 = index * interval / (vnode_count + 1)
    offset := time.Duration(v.Index) * interval / time.Duration(v.Anchor.MaxVNodes+1)
    randomJitter := time.Duration(rand.Intn(jitter)) * time.Second
    time.Sleep(offset + randomJitter)
    ticker := time.NewTicker(interval)
    for range ticker.C {
        v.stabilize() // L1 独立执行, 结果写入 L1 后继列表
        v.fixFingers() // L1 独立执行, RTT 查询经 L0 singleflight 去重
        v.checkPredecessor() // L1 独立执行
    }
}

```

```
}  
}
```

8.7 资源配置参数

参数	层级	默认值	说明
max_vnodes	L0/L1	8	单个 Anchor 最多允许的 vnode 数量
shared_nodeinfo_cache_size	L0	2048	NodeInfo LRU 缓存容量（条目数）
shared_nodeinfo_cache_ttl	L0	30s	NodeInfo 缓存条目 TTL
shared_rtt_cache_ttl	L0	60s	RTT 测量结果缓存 TTL
shared_route_cache_size	L0	1024	路由 LRU 缓存容量（条目数）
shared_route_cache_ttl	L0	10s	路由缓存条目 TTL（拓扑变化窗口）
shared_proof_verify_cache_size	L0	256	VNodeProof 验证结果缓存容量
vnode_goroutine_limit	L1	32	每个 vnode 允许的最大并发 goroutine 数
vnode_maintenance_jitter	L1	5s	维护任务随机抖动范围

8.8 级联故障处理

物理机崩溃时，其 Anchor 与所有 vnode 同时下线。其他节点的后继列表若同时包含同一物理机的多个 vnode，将导致后继链连续失效。应对策略：

- v3.0 的动态后继列表会尽量选择**不同 anchor_id** 的节点填充后继列表（多样性约束，见 8.9）
- L0 NodeInfo 缓存在节点崩溃后，TTL 到期前仍保留旧条目，避免短时间内反复请求已崩溃节点
- 环上其他节点的 stabilize 会在下一个周期内跳过所有失效 vnode，恢复正常

8.9 后继列表多样性约束

约束规则

后继列表中，来自同一 anchor_id 的节点数量不超过 $\text{floor}(\text{successor_list_size} / 2)$ 。此约束由 Anchor 在本地维护后继列表时自动执行，且在 stabilize 过程中对收到的后继列表进行过滤。

第九章：Tracker API 变更规范

9.1 新增：Tracker 注册包含 VNode 列表

POST /tracker/register（变更）

请求体新增字段：

```
{
  "node_info": {
    "node_id": "alc9...",
    "uri": "http://host:port",
    ...
    "vnodes": [
      { "vnode_id": "b3f2...", "index": 1, "proof": {...} },
      { "vnode_id": "d7a1...", "index": 2, "proof": {...} }
    ]
  }
}
```

Tracker 在接受注册前，**对每个 vnode 执行 VNodeProof 验证**。

9.2 新增：Tracker vnode 数量策略

GET /tracker/policy（新增）

返回当前网络的 vnode 策略：

```
{
  "max_vnodes_per_anchor": 8,
  "min_anchor_ratio": 0.3
}
```

min_anchor_ratio：网络中纯锚节点（vnode_count=0）占总物理节点的最低比例，防止全部节点都大量生成 vnode 导致环退化为虚假的“大环”。

9.3 变更：节点列表返回格式

GET /tracker/nodes（变更）

返回结果新增 vnodes 数组。新加入节点收到节点列表后，**仅使用 Anchor 节点（不含 vnode）作为初始 join 目标**，避免 join 到一个立即跟随 Anchor 下线的 vnode 上。

第十章：Node API 变更规范

10.1 所有现有 API 兼容

v4.0 的 vnode 完全实现 v1.0~v3.0 定义的全部 Node API，路径前缀调整为 /chord/node/{node_id}/。

10.2 新增：GET /chord/node/{node_id}/vnode_info

返回该节点的 vnode 元信息（仅 vnode 节点有效，Anchor 返回 404）：

```
{
  "vnode_id": "b3f2...",
  "anchor_id": "a1c9...",
  "index": 1,
  "proof": { ...VNodeProof... },
  "anchor_uri": "http://host:port"
}
```

用于其他节点在路由时判断是否需要附加 VNodeProof 验证。

10.3 新增：GET /chord/node/{anchor_id}/list_vnodes

仅对 Anchor 节点有效，返回其当前所有活跃 vnode 列表：

```
{
  "anchor_id": "a1c9...",
  "vnodes": [
    { "vnode_id": "b3f2...", "index": 1, "status": "ACTIVE" },
    { "vnode_id": "d7a1...", "index": 2, "status": "JOINING" }
  ]
}
```

第十一章：边缘情况与处理策略（36 个场景）

11.1 拓扑与协议边缘

场景 1：两个 sibling vnode 互为后继

同一 Anchor 的 vnode-1 和 vnode-2 在环上相邻，stabilize 后互为后继/前驱。

处理：协议层无需特殊处理，逻辑正确；但运维层面应意识到这意味着所有跨越该区间的键在物理上均在同一台机器，需评估是否调整 vnode 数量。

场景 2：物理机整体崩溃（Anchor + 所有 vnode 同时下线）

后继链中可能连续多个节点失效。

处理：v3.0 快速崩溃检测（check_predecessor 超时）与动态后继列表中的多样性约束共同缓解。其余节点的 stabilize 在 $O(\text{stabilize_interval})$ 时间内恢复连通性。

场景 3：vnode_id 与另一物理节点的 anchor_id 碰撞

SHA1 碰撞概率极低（ 2^{-80} 量级），但需处理。

处理：Tracker 在注册时检测所有已注册 ID（含 vnode_id）是否冲突，冲突则拒绝注册并返回 409 Conflict，错误码 ID_COLLISION。

场景 4: Anchor 主动离网但未停止 vnode

配置错误或 bug 导致 Anchor Chord 节点执行 Graceful Leave，但 vnode 仍然运行。

处理：Anchor 在 Graceful Leave 时，必须先对所有 vnode 执行 Graceful Leave，待全部完成后再执行自身 Leave。
vnode Leave 顺序：index 从大到小（反向离开，减少连锁影响）。

场景 5: Anchor 处于 ISOLATED 状态时 vnode 尝试 Join

Anchor 网络中断，无法联系 Tracker 或其他节点。

处理：vnode Join 依赖 Anchor 提供初始联系点，Anchor 处于 ISOLATED 时，新 vnode 的 Join 阻塞等待，不自行尝试重新加入，避免环境不稳定时产生错误拓扑。

场景 6: vnode 手指表多个条目均指向 sibling vnode

同一 Anchor 的多个 vnode 覆盖了较大 ID 区间时，手指表查询在物理上不离开本机。

处理：无需干预，协议正确；但 v3.0 的 RTT 测量对本机 vnode 的延迟几乎为零，可能导致路由缓存策略倾向于将流量集中在本机，需注意 region-aware 路由不应假设低 RTT 意味着更优选择。

场景 7: 后继列表被同一 Anchor 的多个 vnode 填满

违反 8.5 节多样性约束。

处理：stabilize 时应用多样性过滤，主动丢弃超额的 sibling vnode 条目，替换为其他 anchor 的节点。

场景 8: vnode_count = 0 的纯锚节点（向后兼容）

完全退化为 v3.0 节点，不生成任何 VNodeProof，NodeInfo 中无 vnodes 字段。

处理：接收方遇到无 vnodes 字段的 NodeInfo 时，按 v3.0 流程处理。

场景 9: 稳定运行中新增 vnode（热添加）

运维动态增加 vnode 数量。

处理：新增 vnode 执行正常 Join 流程（与冷启动相同），不影响已运行的 vnode 和 Anchor。需注意 VNodeProof 需提前生成（Anchor 重新签名）再执行 Join。

场景 10: vnode 已离网但 Anchor 手指表仍引用该 vnode_id

Anchor 自身也维护手指表，可能在本机 vnode 离开后仍查询已失效 ID。

处理：vnode 执行 Graceful Leave 时，同步通知 Anchor 清除含该 vnode_id 的手指表条目（标记为无效，触发下次 fix_fingers 时跳过），v3.0 的手指表有效性标记机制可直接复用。

场景 11: 两个 Anchor 独立生成 index=1 的 vnode，ID 不同（正常情况）

由于 anchor_id 不同，SHA1 输入不同，vnode_id 天然不同，无冲突。属于正常情况，记录此场景以确认设计的正确性。

场景 12: stabilize 运行时，前驱检测发现前驱是自身的 sibling vnode

vnode-1 的前驱是 vnode-2（同一 Anchor），需要正确处理 notify 流程。

处理：协议层视其为普通节点，notify 正常执行；运维层记录警告日志，提示该 Anchor 的 vnode 在环上聚集，建议调整 vnode 数量。

场景 13: 环中只有 1 个物理节点，但包含 N 个 vnode

所有 Chord 交互均在本机完成，后继/前驱关系构成本机内环。

处理：协议层正确运行（环路由退化为本地调用），需注意请求签名验证逻辑在自签名情况下不应短路，保持代码路径一致性。

场景 14：vnode 后继列表跨越多个 Anchor

正常且期望的状态，后继列表多样性约束的目标之一。

处理：无特殊处理，记录此场景为健康状态的参照基准。

场景 15：fix_fingers 并行修复时发现多个条目同时指向同一 sibling vnode

并行修复（v3.0 机制）无法分散 sibling 聚集问题。

处理：在 fix_fingers 结果写入前，应用 sibling 去重策略：若手指表中来自同一 anchor_id 的条目超过阈值（默认 $\text{floor}(m/4)$ ），对其中的冗余条目标记为待更新，在下个周期优先修复这些条目。

场景 16：vnode Join 时使用了另一个 sibling vnode 作为 Bootstrap 节点

新 vnode 通过 Anchor 的 list_vnodes 接口发现已有 sibling，并尝试以 sibling 作为 Join 入口。

处理：允许此操作（sibling 是合法 Chord 节点），但建议优先使用 Tracker 提供的非 sibling 节点作为 Bootstrap，减少 ID 聚集。

11.2 VNodeProof 生命周期边缘

场景 17：VNodeProof 在请求传输过程中到期

请求发出时 proof 有效，到达时已过期。

处理：接收方允许 clock_skew_tolerance（默认 30 秒）的容差，超出则返回 403，错误码 PROOF_EXPIRED，要求重试。发送方在每次请求前检查 proof 剩余有效期，若不足 vnodeproof_renew_before，先重签再发送。

场景 18：Anchor 私钥轮换后旧 VNodeProof 仍在流通

v2.0 支持密钥轮换。轮换后，旧 proof 的签名无法用新公钥验证。

处理：密钥轮换时，Anchor 立即为所有 vnode 重签 VNodeProof，并在轮换后的 key_rotation_grace_period（默认 300 秒）内同时接受新旧公钥签名的 proof。

场景 19：Anchor CA 证书到期但 VNodeProof 仍有效

CA 证书过期导致 anchor 公钥无法验证，连带使所有 vnode 的 VNodeProof 验证失败。

处理：CA 证书更新时，同步触发所有 vnode 的 VNodeProof 重签（因为 proof 中的 anchor_pub 通过 CA 证书验证链传递）。Anchor 应在 CA 证书到期前主动续期。

场景 20：vnode 重启后 Anchor 尚未完成重签

vnode 持有旧 VNodeProof 尝试重新 Join，而 Anchor 在重启过程中尚未完成 proof 生成。

处理：vnode Join 前必须持有有效 VNodeProof，否则阻塞至 Anchor 提供后再 Join。Anchor 负责在自身完成 ACTIVE 状态前，为所有 vnode 预签 proof。

第十二章：安全攻击面分析

12.1 Sybil 攻击（过度生成 vnode）

攻击描述：恶意 Anchor 将 max_vnodes 配置为极大值，在环上占据大量 ID 点，控制大部分键空间，实施路由劫持或键丢弃攻击。

缓解措施：

- Tracker 在注册时强制执行 max_vnodes_per_anchor 策略限制，拒绝超额 vnode 的注册请求
- CA 在签发证书时可在证书扩展字段中写入 max_vnodes 授权值，节点无法声明超过授权值的 vnode
- Tracker 的 min_anchor_ratio 约束确保网络中有足够多的纯物理节点
- 其他节点对来自同一 anchor_id 的节点在路由时保持多样性，不允许同一 anchor 的 vnode 占据路由路径上的连续跳数超过阈值（默认 2 跳）

12.2 VNodeProof 伪造

攻击描述：攻击者尝试伪造 VNodeProof，声称某个 vnode_id 属于合法 Anchor，绕过证书验证。

缓解措施：

- VNodeProof 由 Anchor Ed25519 私钥签名，私钥不出本机，无法在不拥有私钥的情况下伪造
- 签名覆盖 vnode_id、anchor_id、index、时间戳，任何字段篡改均导致签名验证失败
- anchor_pub 的合法性通过 CA 证书验证链保障，攻击者无法伪造 CA 证书

12.3 VNodeProof 重放攻击

攻击描述：攻击者截获有效 VNodeProof，在其有效期内对不同请求重放，冒充该 vnode 的身份。

缓解措施：

- VNodeProof 本身不防重放（它仅证明 vnode 身份，不绑定具体请求）
- 请求防重放依赖 v2.0 的 X-Chord-Signature 机制（含 nonce + timestamp），VNodeProof 与请求签名共同使用才能完成完整验证
- 攻击者即便持有 VNodeProof，仍无法伪造请求签名（需要 Anchor 私钥）

12.4 anchor_id 声明伪造

攻击描述：攻击者在请求 Header 中填入他人的 anchor_id，试图借用合法 Anchor 的身份为自己的 vnode 背书。

缓解措施：

- 接收方验证 anchor_id 的步骤：从请求 Header 获取 anchor_id → 获取该 anchor 的 CA 证书 → 用 CA 证书中的公钥验证请求签名 → 验证 VNodeProof 签名
- 攻击者不持有合法 Anchor 的私钥，无法通过请求签名验证
- anchor_id 绑定于 CA 证书中的 URI 字段，无法声明他人的 anchor_id

12.5 ID 位置操纵（Index 选择攻击）

攻击描述：攻击者通过遍历 index，选择特定 index 使 vnode_id 落在目标键的附近，控制特定键范围。

缓解措施：

- max_vnodes 限制了可尝试的 index 数量（最多 8 个），攻击者无法大范围遍历
- 即便攻击者找到有利 index，也需要获得该 index 的授权（若 CA 证书记录 max_vnodes，则 index 必须在 [1, max_vnodes] 范围内，选择空间有限）
- 相比直接控制物理节点 URI 进行 SHA1 碰撞（v1.0 的攻击面），vnode 的 ID 操纵空间更小

12.6 Eclipse 攻击（vnode 围攻目标键）

攻击描述：攻击者通过精心选择多个 Anchor 的 vnode 配置，使目标键的前后相邻节点全部为攻击者控制的 vnode，完全截断该键的路由。

缓解措施：

- max_vnodes 和 min_anchor_ratio 限制了单个攻击者的环覆盖比例
- 路由多样性要求（不允许同 anchor 的 vnode 连续占据路由跳）使纯 vnode eclipse 更难实现
- 与原始 Chord 安全性相同：需要控制相当比例的节点，难度随网络规模增加

12.7 VNodeProof 过期检查绕过

攻击描述：攻击者构造接收方，跳过 expires_at 检查，永久接受旧 proof。

缓解措施：这属于实现层漏洞，通过代码审查和测试覆盖防范；规范明确要求 expires_at > now，测试用例覆盖过期 proof 应返回 403。

12.8 时钟偏移滥用

攻击描述：攻击者将本机时钟拨快，使过期 proof 在 clock_skew_tolerance 窗口内仍被接受，延长攻击窗口。

缓解措施：

- clock_skew_tolerance 设定为保守值（30 秒），限制可利用的窗口
- 接收方以自身时钟为基准判断，不信任发送方的时间声明
- 系统时间同步（NTP）为基础防护建议

12.9 证书吊销未传播

攻击描述：CA 吊销某 Anchor 证书后，已持有该 Anchor 签名 VNodeProof 的节点仍被其他节点信任，因为证书吊销信息尚未传播到所有节点。

缓解措施：

- 继承 v2.0 的证书吊销机制，Tracker 维护 CRL（证书吊销列表），节点定期拉取（间隔 crl_refresh_interval，默认 300 秒）
- 吊销证书的 anchor 的所有 vnode 的 VNodeProof 在 CRL 更新后自动失效（验证链断裂）

- Tracker 接收到吊销通知后立即移除该 anchor 及其所有 vnode 的注册信息，并通过主动推送通知在线节点（若实现了推送机制）

12.10 DoS（针对单一 vnode 的维护消耗）

攻击描述：攻击者向某个 vnode 持续发送 stabilize 或 notify 请求，消耗该 vnode 的处理资源，进而影响同一物理机上的所有 vnode。

缓解措施：

- HTTP Server 层实现速率限制（rate limiting），按 caller node_id 或 IP 进行限流
- 请求签名验证确保匿名请求无法通过，限制了 DoS 的来源
- 物理机级别的 goroutine 上限（vnode_goroutine_limit）防止单个 vnode 耗尽系统资源

12.11 vnode 非授权 Graceful Leave 注入

攻击描述：攻击者向某 vnode 发送伪造的 Graceful Leave 指令，使其主动离开网络（若存在外部触发 Leave 的 API）。

缓解措施：Graceful Leave 仅由本机内部逻辑触发，不通过 RESTful API 对外暴露触发接口，外部节点无法触发他人的 Leave。

12.12 跨版本协议降级攻击

攻击描述：攻击者伪装为 v3.0 节点，使 v4.0 节点退回不需要 VNodeProof 的验证路径，绕过 vnode 身份验证。

缓解措施：

- 节点在 NodeInfo 中声明协议版本（"protocol_version": "4.0"）
- v4.0 节点收到来自声称 v4.0 协议版本的请求时，强制要求完整的 v4.0 验证路径
- 降级只允许发生在请求方声明低版本时，不允许通过省略 VNodeProof Header 来触发降级

第十三章：新增配置参数

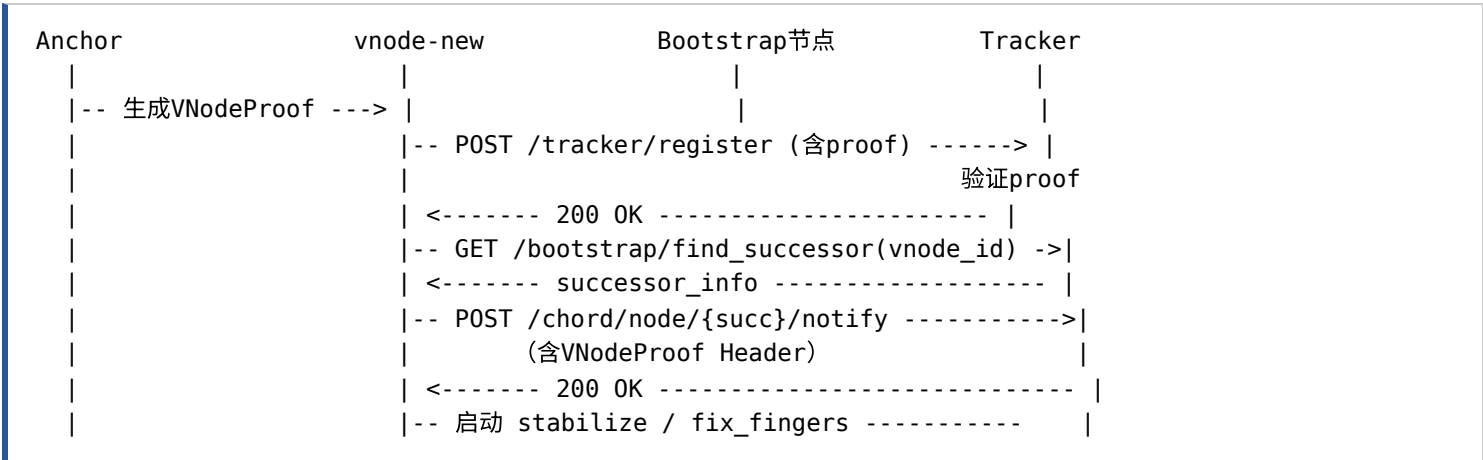
以下所有参数均在节点配置文件中设定：

参数名	类型	默认值	说明
vnode_count	int	0	该物理节点生成的 vnode 数量，0 表示纯锚节点模式
max_vnodes	int	8	全局限制（Tracker 策略），单个 Anchor 最多允许的 vnode 数
vnodeproof_ttl	int (秒)	86400	VNodeProof 有效期（秒）
vnodeproof_renew_before	int (秒)	3600	到期前多少秒触发自动重签

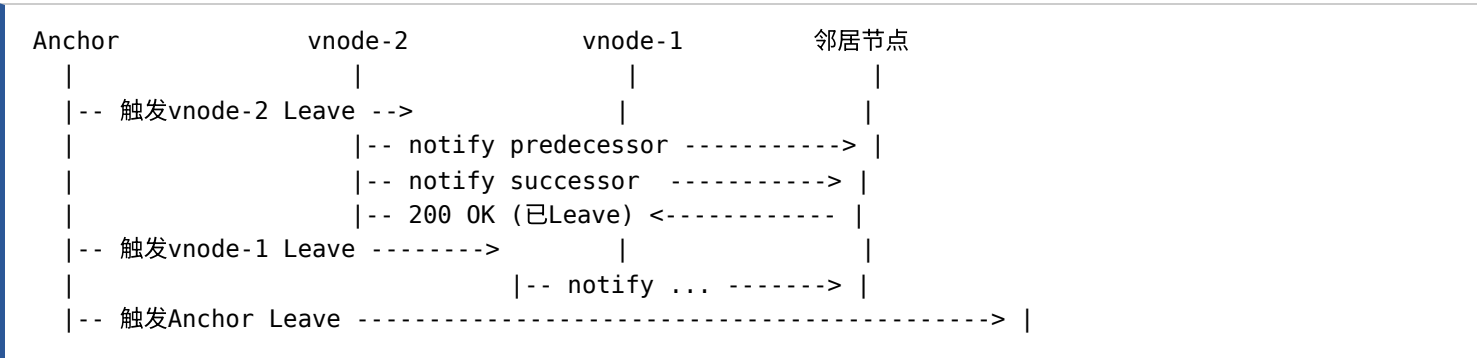
参数名	类型	默认值	说明
clock_skew_tolerance	int (秒)	30	接受 VNodeProof 的时钟偏差容差
vnode_goroutine_limit	int	32	每个 vnode 的最大并发 goroutine 数
vnode_maintenance_jitter	int (秒)	5	维护任务随机抖动范围
sibling_route_max_hops	int	2	路由路径上允许连续经过同 anchor vnode 的最大跳数
successor_list_sibling_cap	float	0.5	后继列表中同 anchor 节点的最大占比
crl_refresh_interval	int (秒)	300	证书吊销列表刷新闻隔
min_anchor_ratio	float	0.3	网络中纯锚节点最低占比（Tracker 策略）
vnode_bootstrap_prefer_external	bool	true	Join 时优先使用非 sibling 节点作为 Bootstrap
shared_nodeinfo_cache_size	int	2048	L0 NodeInfo TTL 缓存容量（条目数）
shared_nodeinfo_cache_ttl	int (秒)	30	L0 NodeInfo 缓存条目 TTL
shared_rtt_cache_ttl	int (秒)	60	L0 RTT 测量结果缓存 TTL
shared_route_cache_size	int	1024	L0 路由 LRU 缓存容量（条目数）
shared_route_cache_ttl	int (秒)	10	L0 路由缓存条目 TTL
shared_proof_verify_cache_size	int	256	L0 VNodeProof 验证结果缓存容量
transfer_timeout	int (秒)	30	数据迁移 transfer_ack 等待超时

第十四章：消息时序变更

14.1 新 vnode 加入网络时序



14.2 物理机整体下线时序（主动）



14.3 请求签名 + VNodeProof 组合验证时序



第十五章：实现建议（Go 语言）

15.1 数据结构

```
// VNodeProof 凭证
type VNodeProof struct {
    VNodeID    string `json:"vnode_id"`
    AnchorID   string `json:"anchor_id"`
    Index      int    `json:"index"`
    IssuedAt   int64  `json:"issued_at"`
    ExpiresAt  int64  `json:"expires_at"`
    AnchorPub  string `json:"anchor_pub"` // Base64(ed25519公钥)
    Signature  string `json:"signature"`  // Base64(ed25519签名)
```

```

}

// Anchor 维护的 vnode 集合
type AnchorNode struct {
    ChordNode           // 继承 v3.0 节点结构
    PrivateKey ed25519.PrivateKey
    VNodes      map[string]*VNode // vnode_id -> VNode
    MaxVNodes   int
    Dispatcher  *NodeDispatcher
}

// VNode 结构
type VNode struct {
    ChordNode           // 完整的 Chord 节点状态
    Anchor *AnchorNode
    Index   int
    Proof   *VNodeProof
    ProofMu sync.RWMutex // 保护 Proof 的并发读写
}

// 路由分派器
type NodeDispatcher struct {
    anchor *AnchorNode
    vnodes map[string]*VNode // vnode_id -> VNode
    mu     sync.RWMutex
    mux    *http.ServeMux
}

```

15.2 vnode_id 派生函数

```

func DeriveVNodeID(anchorID string, index int) string {
    input := fmt.Sprintf("chord-vnode-v4\n%s\n%d", anchorID, index)
    h := sha1.Sum([]byte(input))
    return hex.EncodeToString(h[:])
}

```

15.3 VNodeProof 签名与验证

```

func (a *AnchorNode) SignVNodeProof(index int, ttl time.Duration) (*VNodeProof, error) {
    vnodeID := DeriveVNodeID(a.ID, index)
    now := time.Now().Unix()
    proof := &VNodeProof{
        VNodeID:  vnodeID,
        AnchorID: a.ID,
        Index:    index,
        IssuedAt: now,
        ExpiresAt: now + int64(ttl.Seconds()),
        AnchorPub: base64.StdEncoding.EncodeToString(a.PrivateKey.Public().(ed25519.PublicKey)),
    }
    canonical := buildProofCanonical(proof)
    sig := ed25519.Sign(a.PrivateKey, []byte(canonical))
}

```

```

    proof.Signature = base64.StdEncoding.EncodeToString(sig)
    return proof, nil
}

func VerifyVNodeProof(proof *VNodeProof, anchorPubKey ed25519.PublicKey, clockSkew time.Duration)
error {
    now := time.Now().Unix()
    if proof.ExpiresAt+int64(clockSkew.Seconds()) < now {
        return ErrProofExpired
    }
    expected := DeriveVNodeID(proof.AnchorID, proof.Index)
    if expected != proof.VNodeID {
        return ErrVNodeIDMismatch
    }
    canonical := buildProofCanonical(proof)
    sig, err := base64.StdEncoding.DecodeString(proof.Signature)
    if err != nil || !ed25519.Verify(anchorPubKey, []byte(canonical), sig) {
        return ErrInvalidProofSignature
    }
    return nil
}

func buildProofCanonical(p *VNodeProof) string {
    return fmt.Sprintf("chord-vnode-proof-v4\n%s\n%s\n%d\n%d\n%d",
        p.VNodeID, p.AnchorID, p.Index, p.IssuedAt, p.ExpiresAt)
}

```

15.4 错峰维护调度

```

func (v *VNode) startMaintenance(interval time.Duration, jitter int) {
    offset := time.Duration(v.Index) * interval / time.Duration(v.Anchor.MaxVNodes+1)
    randomJitter := time.Duration(rand.Intn(jitter)) * time.Second
    time.Sleep(offset + randomJitter)

    ticker := time.NewTicker(interval)
    for range ticker.C {
        v.stabilize()
        v.fixFingers()           // v3.0 并行修复
        v.checkPredecessor()
    }
}

```

15.5 HTTP 路由注册

```

func (d *NodeDispatcher) RegisterRoutes() {
    // 为 Anchor 注册路由
    d.registerNodeRoutes(d.anchor.ID, d.anchor.ChordNode)
    // 为每个 vnode 注册路由
    d.mu.RLock()
    defer d.mu.RUnlock()
    for id, vn := range d.vnodes {

```

```
        d.registerNodeRoutes(id, vn.ChordNode)
    }
    // 向后兼容路径（无 node_id 前缀，默认路由到 Anchor）
    d.mux.Handle("/find_successor", d.anchor.handleFindSuccessor())
    d.mux.Handle("/notify",          d.anchor.handleNotify())
}

func (d *NodeDispatcher) registerNodeRoutes(nodeID string, node ChordNode) {
    base := "/chord/node/" + nodeID
    d.mux.Handle(base+"/find_successor",  vnodeProofMiddleware(node.handleFindSuccessor()))
    d.mux.Handle(base+"/notify",          vnodeProofMiddleware(node.handleNotify()))
    d.mux.Handle(base+"/get_predecessor", vnodeProofMiddleware(node.handleGetPredecessor()))
    d.mux.Handle(base+"/ping",            vnodeProofMiddleware(node.handlePing()))
    // ...其余 API
}
```

第十六章：数据迁移时序

16.1 概述与协议约定

Chord 的键责任原则：节点 N 负责区间 $[predecessor.id, N.id]$ 内的所有键。当环拓扑发生变化时（vnode 加入或离开），键的责任区间随之转移，数据层需要相应地迁移键值对。

学习系统说明

v4.0 基础系统本身不实现数据存储层（仅实现 Chord 路由协议）。本章描述**当上层应用或数据层存在时**，正确实现键迁移的协议行为与时序规范。

迁移场景共四类：

编号	场景	迁移方向	数据完整性保证
T1	新 vnode 加入	Successor → 新 VNode（接收一部分键）	两阶段确认，Successor 在 ACK 前保留副本
T2	VNode 主动离开	离开 VNode → Successor（转移所有键）	转移完成后再执行 Graceful Leave
T3	物理机整体崩溃	无（数据丢失，依赖副本层恢复）	副本层恢复（本章仅描述协议行为）
T4	有序关机（Graceful Shutdown）	反向 index 顺序逐 vnode 迁移	避免数据黑洞，逐步无损转移

16.2 新增迁移 API

POST /chord/node/{node_id}/transfer_keys

由发送方调用，推送键值对到接收方。接收方逐批写入本地存储并返回已接收数量。

```
// 请求体
{
  "from_id": "b3f2...",
  "range_lo": "a1c9...",
  "range_hi": "b3f2...",
  "entries": [
    { "key": "SHA1_hex", "value": "Base64(data)", "version": 42 }
  ],
  "batch_seq": 1,
  "is_last": true
}
// 响应体
{ "received": 128, "batch_seq": 1, "ack_token": "nonce" }
```

POST /chord/node/{node_id}/transfer_ack

由接收方调用，通知发送方所有批次已成功写入，发送方可安全删除已迁移键的副本。

```
// 请求体
{ "from_id": "b3f2...", "range_lo": "a1c9...", "range_hi": "b3f2...",
  "total_keys": 128, "ack_token": "与首批次返回的 token 对应" }
// 响应体
{ "status": "ok" }
```

16.3 T1：新 VNode 加入时的键接收时序

新 vnode 在完成 Join（确认 Successor 和 Predecessor）后，向 Successor 发起键拉取请求，进入两阶段确认流程。

新 VNode (vnode-new)	Successor (succ)
-- JOIN 完成，确认 succ 与 pred -----	
-- POST /transfer_keys -----	
{ range_lo:pred.id, range_hi:self.id,	--- 查询本地存储，找出区间内所有键
request_keys:true }	
<-- { entries:[...], batch_seq:1, is_last:false }	
<-- { entries:[...], batch_seq:2, is_last:true }	
--- 写入本地存储（逐批确认） -----	
-- POST /transfer_ack -----	
{ total_keys:256, ack_token:"..." }	--- 收到 ACK 后删除已转移键的副本
<-- { status:"ok" } -----	
--- 状态切换为 ACTIVE，开始正常服务 -----	

关键约束

Successor 在收到 transfer_ack 之前，必须保留这些键的副本。若 vnode-new 在接收过程中崩溃，Successor 仍持有数据不丢失。超过 transfer_timeout（默认 30 秒）未收到 ACK，Successor 保持键不变，等待 vnode-new 重启后重新发起迁移。

16.4 T2: VNode 主动离开时的键转移时序

VNode 执行 Graceful Leave 时，在通知邻居节点更新路由之前，先完成键迁移。若直接 Successor 是 sibling vnode，必须沿后继链找到第一个 anchor_id 不同的外部节点。

离开 VNode (vnode-leave)	Sibling 检查	外部 Successor (real_succ)
-- succ.anchor_id == self.anchor_id?		
YES: 沿链找下一个非 sibling		
NO: 使用当前 succ -----		
-- POST /transfer_keys → real_succ -----		
{ all keys in (pred.id, self.id] }		--- 逐批写入
<-- 200 OK (逐批确认) -----		
-- POST /transfer_ack → real_succ -----		
<-- { status:"ok" } -----		
--- 键迁移完成, 开始 Graceful Leave -----		
-- notify predecessor: "succ = real_succ" -----		
-- notify successor: "pred = my_pred" -----		
--- 状态变为 LEAVING, 向 Tracker 注销 -----		

Sibling Successor 必须跳过

若 vnode-leave 的直接 Successor 是同一物理机的 sibling vnode，该 sibling 也即将随关机流程下线，将键迁移给它会导致数据丢失。必须沿后继链找到第一个 anchor_id 不同的节点作为迁移目标。

16.5 T3: 物理机整体崩溃时的路由恢复

物理机崩溃时，Anchor 与所有 vnode 同时下线，无法执行主动迁移。键数据随内存丢失（若未持久化）。

崩溃物理机	存活节点 -A	存活节点 -B
X <-- 崩溃		
	-- check_predecessor → X (超时)	
	--- 前驱标记为失效 -----	
	--- stabilize: 后继链断裂, 跳过所有崩溃 vnode	
	--- 更新后继为崩溃物理机的下一个存活节点	
	路由收敛 (0(stabilize_interval) 时间内完成)	
	崩溃 vnode 的键区间现由其存活 Successor 负责	
	(键数据已丢失; 若有副本层, 则从副本节点恢复)	


```
"redirect_to": { "node_id": "succ_id", "uri": "http://succ:port" }  
}
```

文档结束 | Chord DHT VNode 扩展设计文档 v4.0 | 创建日期: 2026年5月30日

本文档为学习用途设计文档，所有方案均以可读性与可理解性为优先原则，不适合直接用于生产环境。